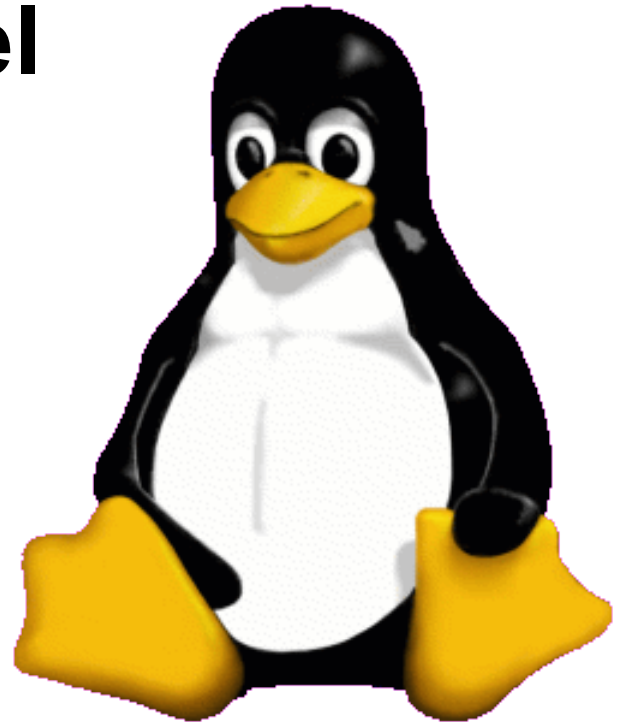


CS353 Linux Kernel

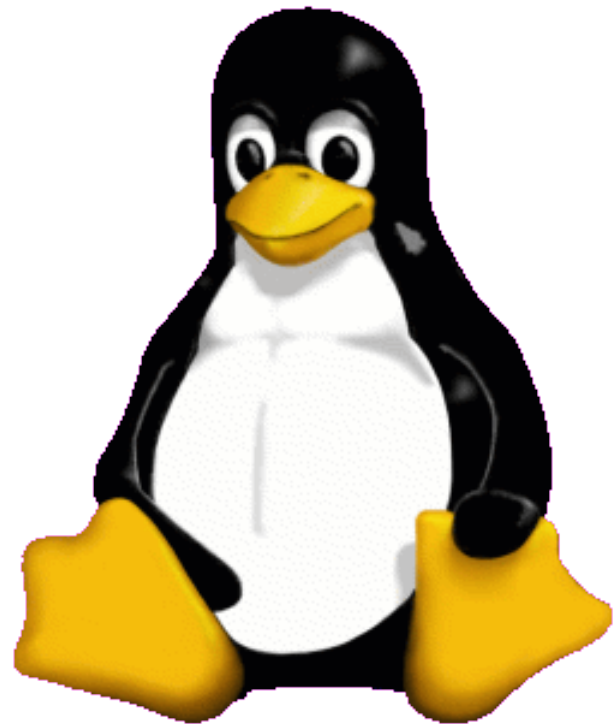
Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn



上海交通大學

Device Drivers

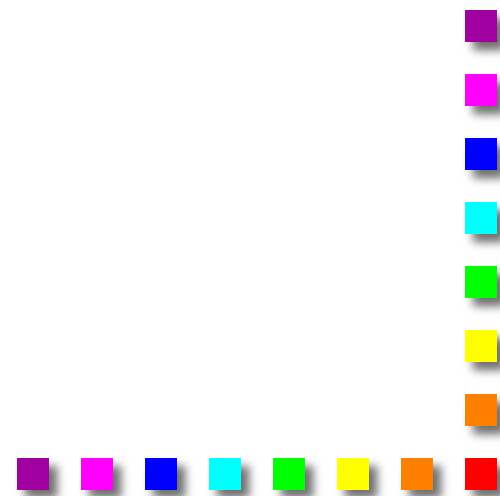
Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn



上海交通大學

Outline

- What are device drivers
- Character Device Drivers
- Block Device Drivers
- DMA
- Network Device Drivers
- Timer



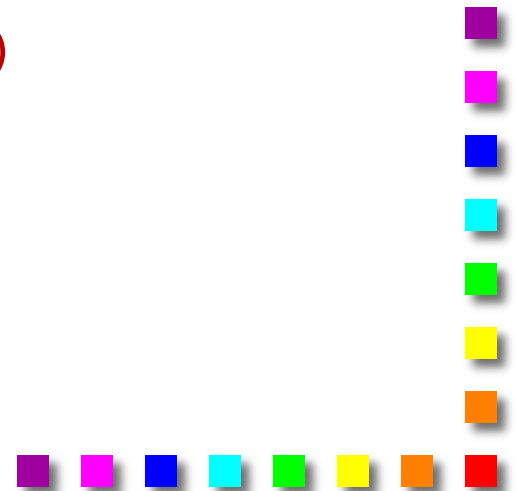
What does a Device Driver do?

- A set of routines that **communicate with a hardware device** and provide a uniform interface to the operating system kernel
- A **self-contained** component that can be added to, or removed from, the operating system dynamically.
- Management of data flow and control between user programs and a peripheral device.
- A **user-defined section of the kernel** that allows a program or a peripheral device to appear as a “/dev” device to the rest of the system’s software.



Within the Kernel

- Device driver resides in the kernel
 - Service interrupts
 - Access device hardware
- Device driver has two sections
 - Interrupt section (real time events)
 - Synchronous section (process must be exec)
- What happens to requesting process?
 - `interruptible_sleep_on(&dev_wait_queue)`
 - `wake_up_interruptible(&dev_wait_queue)`
- Synchronization
 - `cli()` // clear interrupts
 - Critical Section Operations*
 - `sti ()` // set interrupt enable



File Operations

- Devices are accessed as **files**
 - ❑ Nodes of the filesystem tree
 - ❑ Conventionally located in the **/dev** directory
- Applications use standard system calls to open, read from, write to and close them as if the device were a file.
- Registers by adding an entry into **chrdevs vector (character device)**
- Device's **major device identifier** is used as an index into this vector.
 - ❑ For example: 4 for the tty device
 - ❑ Major number for a device is fixed.



Types

■ Character Devices

- Allows **serial access** of data bytes
- Mice, Keyboard, Serial Port, etc

■ Block Devices

- Transfers a block of bytes as a unit
- Allows **random access** to independent, fixed sized blocks of data
- Hard drive, cd-rom, etc

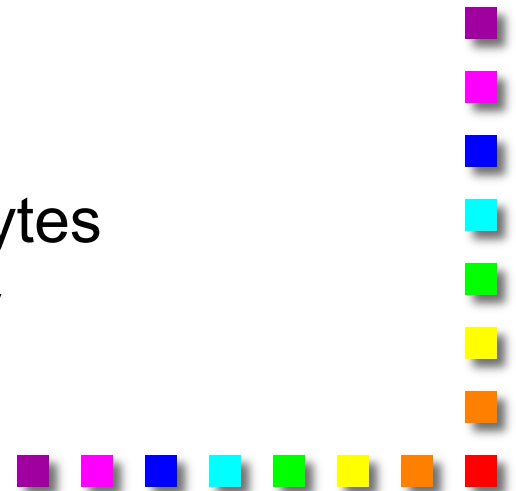
■ Network Devices

- Dealt differently from the above two
- Users **cannot directly transfer data** to network devices
- Communicate indirectly by opening a connection to the kernel's networking system.



Device Controller

- It is a collection of electronics that can operate a port, a bus or a device.
- I/O devices have components:
 - Mechanical component
 - Electronic component → Device Controller
- Task
 - Convert serial bit stream to block of bytes
 - Perform error correction as necessary



How do Device drivers access the Controller

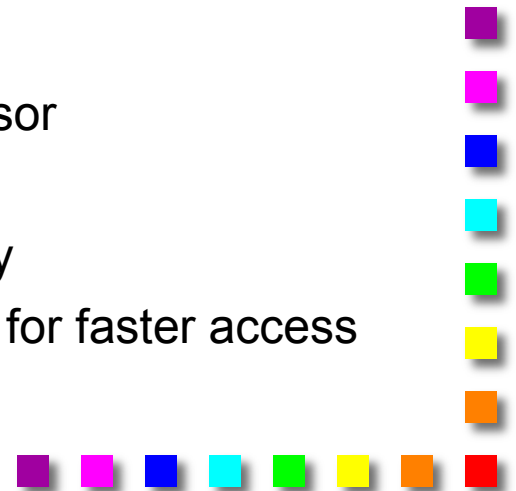
- By reading and writing bit patterns in specific registers of the controller.

- Special I/O Instructions

- Triggers bus lines to select the proper device and to move bits into/out of a device register.
- Valid only in kernel mode, **no longer popular**

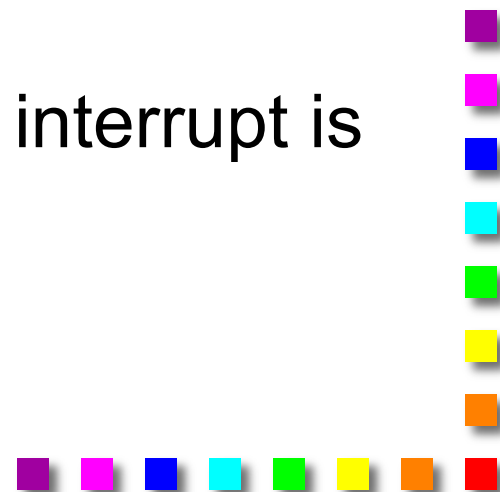
- Memory-mapped I/O

- Registers mapped to address space of processor
- Read and write to special memory addresses
- Protect by placing in kernel address space only
- May map part of device in user address space for faster access



Interrupt

- Device raises an interrupt when it needs to be serviced
- Interrupts being used - /proc/interrupts
- Types
 - Fixed, Floppy Disk Controller always uses interrupt 6
 - Allocated at boot time, PCI interrupts
- Other interrupts stopped when an interrupt is delivered



Interrupts cont...

- Earlier
 - 16 interrupt lines
 - One processor to deal with them
- Modern hardware
 - more interrupts,
 - equipped with advanced programmable interrupt controllers (APICs)
 - can distribute interrupts across multiple processors in an intelligent (and programmable) way.



Interrupt driven I/O

Semantics for generating Interrupts

■ Input:

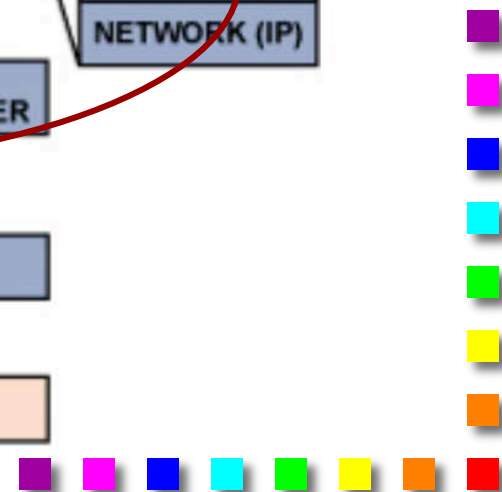
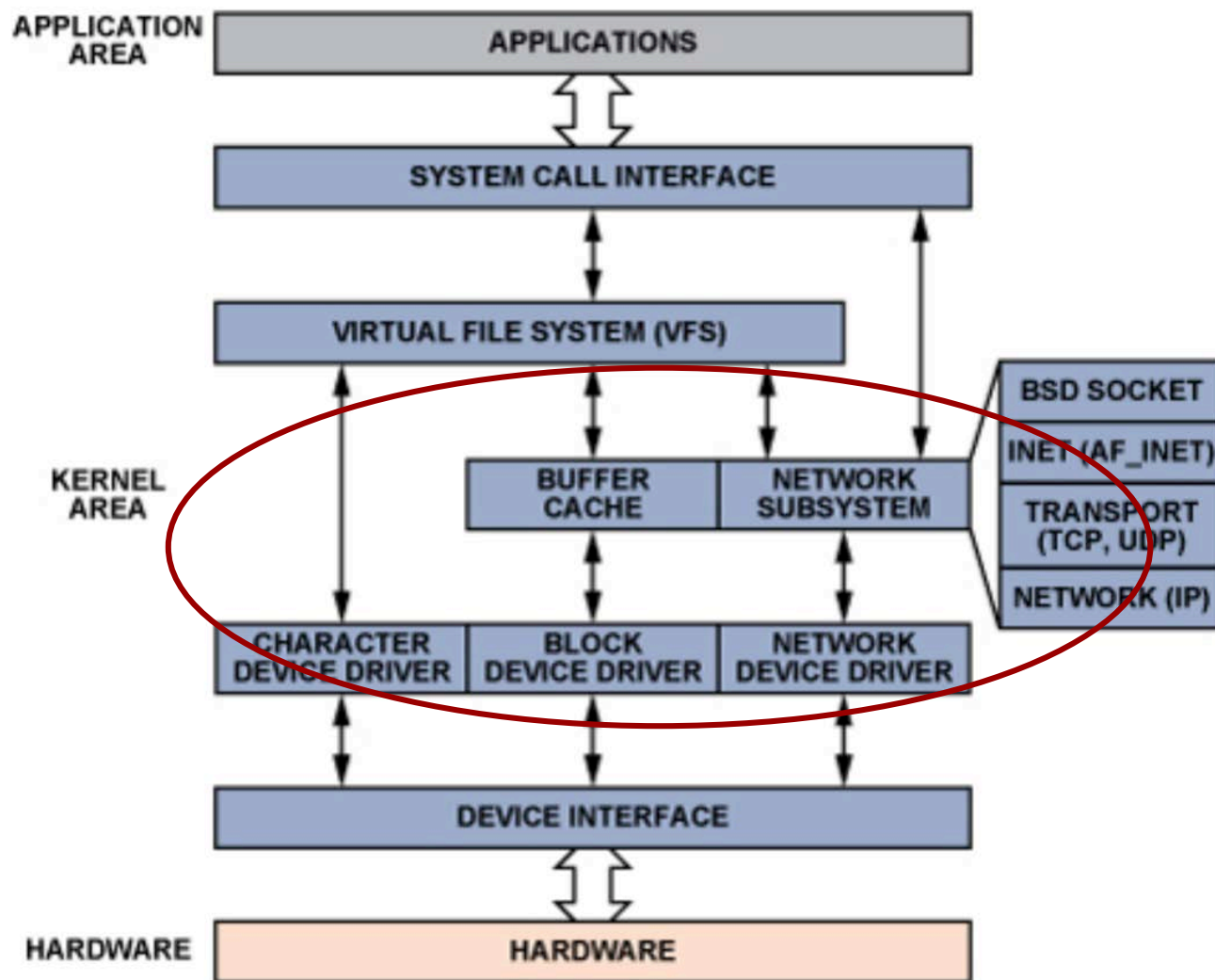
- a) device interrupts the processor when new data has arrived
- b) actual actions to perform depend on whether the device uses I/O ports, memory mapping, or DMA.

■ Output:

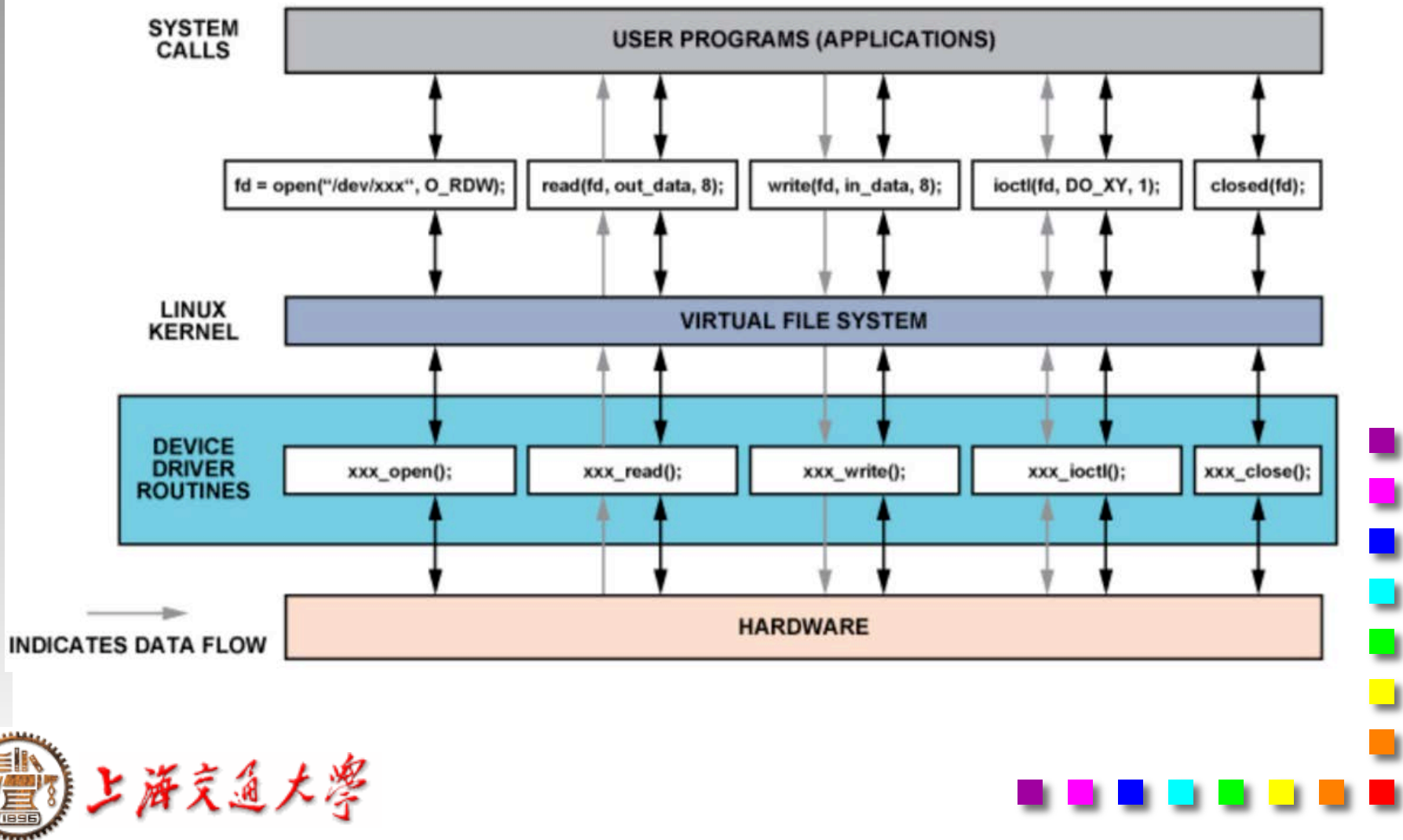
- a) device delivers interrupt when ready to accept new data or to acknowledge a successful data transfer.
- b) Memory-mapped and DMA-capable devices usually generate interrupts to tell the system they are done with the buffer.



Device Driver Interface

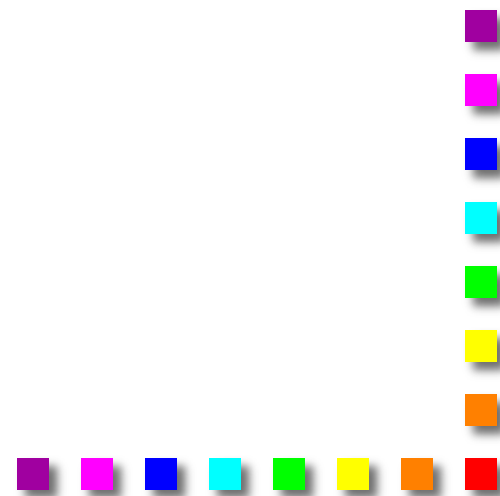


Device Driver Interface



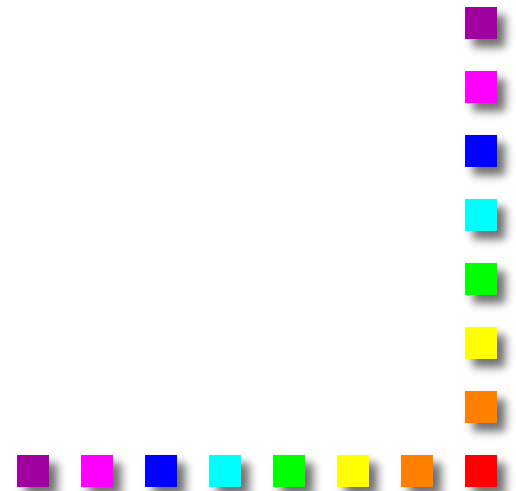
Outline

- What are Device Drivers
- Character Device Drivers
- Block Device Drivers
- DMA
- Network Device Drivers
- Timer



What is a character device

- The simplest of Linux's devices
- Transfers bytes **one by one** (compare with block)
- Referenced by standard system call (get() , put())
 - Like open, read, close, etc
- Standard examples
 - virtual terminals (ttys)
 - serial port
 - keyboard
 - sound



'ls -l' in /dev

Char Device

Major Num

Minor Num

```
crw-rw-rw-  1 root tty      5,   0 Jun  6 16:26 tty
crw--w----  1 root tty      4,   0 Oct 21  2017 tty0
crw--w----  1 root tty      4,   1 Oct 21  2017 tty1
crw--w----  1 root tty      4,  10 Oct 21  2017 tty10
crw--w----  1 root tty      4,  11 Oct 21  2017 tty11
crw--w----  1 root tty      4,  12 Oct 21  2017 tty12
crw--w----  1 root tty      4,  13 Oct 21  2017 tty13
crw--w----  1 root tty      4,  14 Oct 21  2017 tty14
crw--w----  1 root tty      4,  15 Oct 21  2017 tty15
crw--w----  1 root tty      4,  16 Oct 21  2017 tty16
crw--w----  1 root tty      4,  17 Oct 21  2017 tty17
```

- The **major number** identifies the driver associated with the device
- Driver can control multiple devices => **minor number** used to differentiate among them.



上海交通大学



Registering a char device

■ Registering

□ int **register_chrdev** (unsigned int major, const char *name, struct file_operations *fops);

■ Removing a device

□ int **unregister_chrdev** (unsigned int major, const char *name);

■ Create a device node on a file system

mknod /dev/scull0 c 254 0

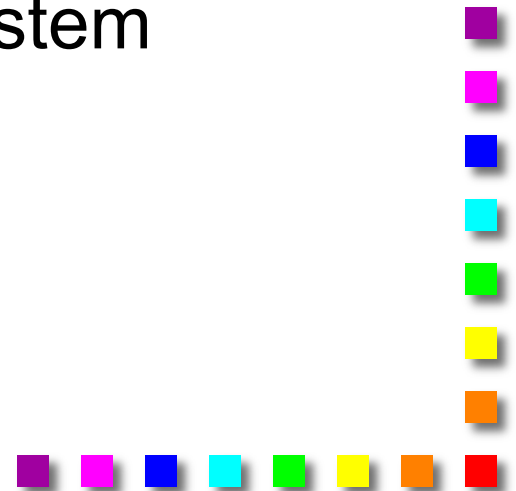
Char device

Major No

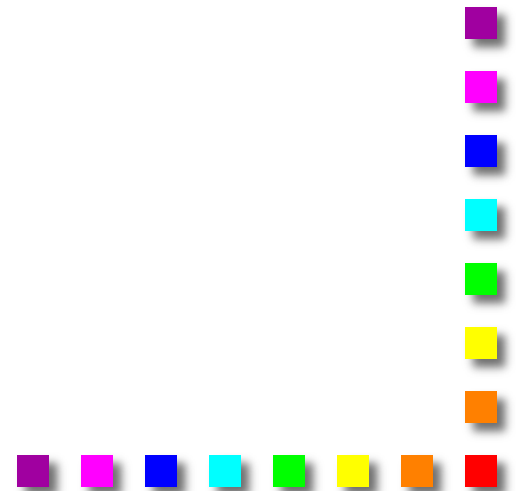
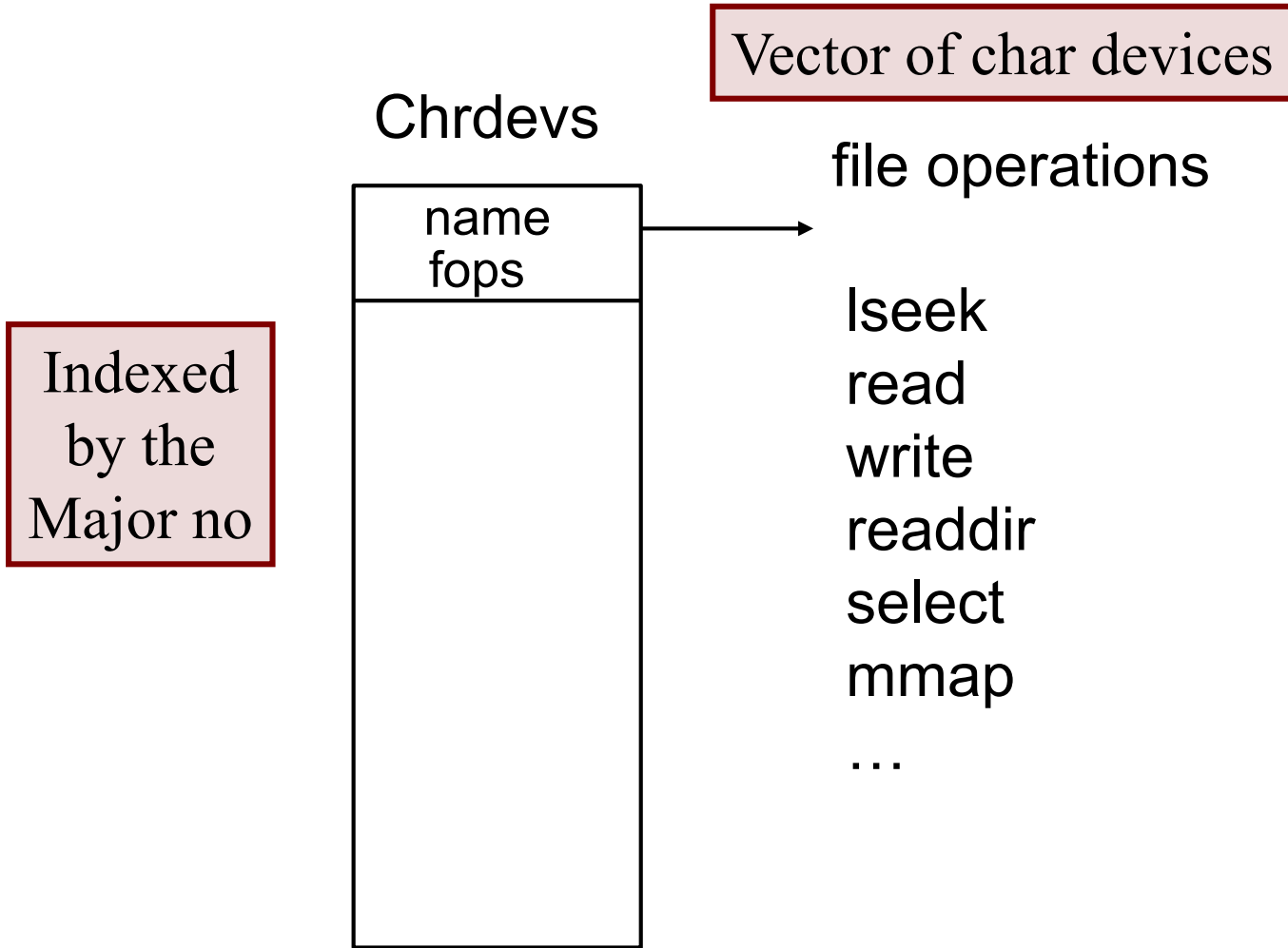
Minor No



上海交通大学



File operations



File operations...

Pointer to

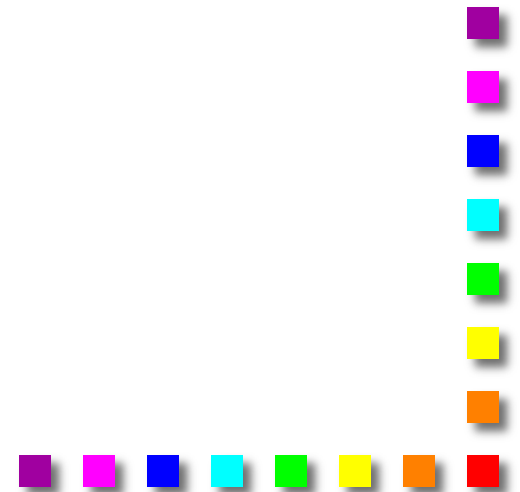
struct file_operations

```
{  
    int (*lseek)(...);  
    int (*read)(...);  
    int (*write)(...);  
    int (*select)(...);  
    int (*ioctl)(...) . . .  
    int (*open)(...);  
    int (*release)(...);  
    . . .  
};
```

Array of function pointers
or
Set as NULL



上海交通大学

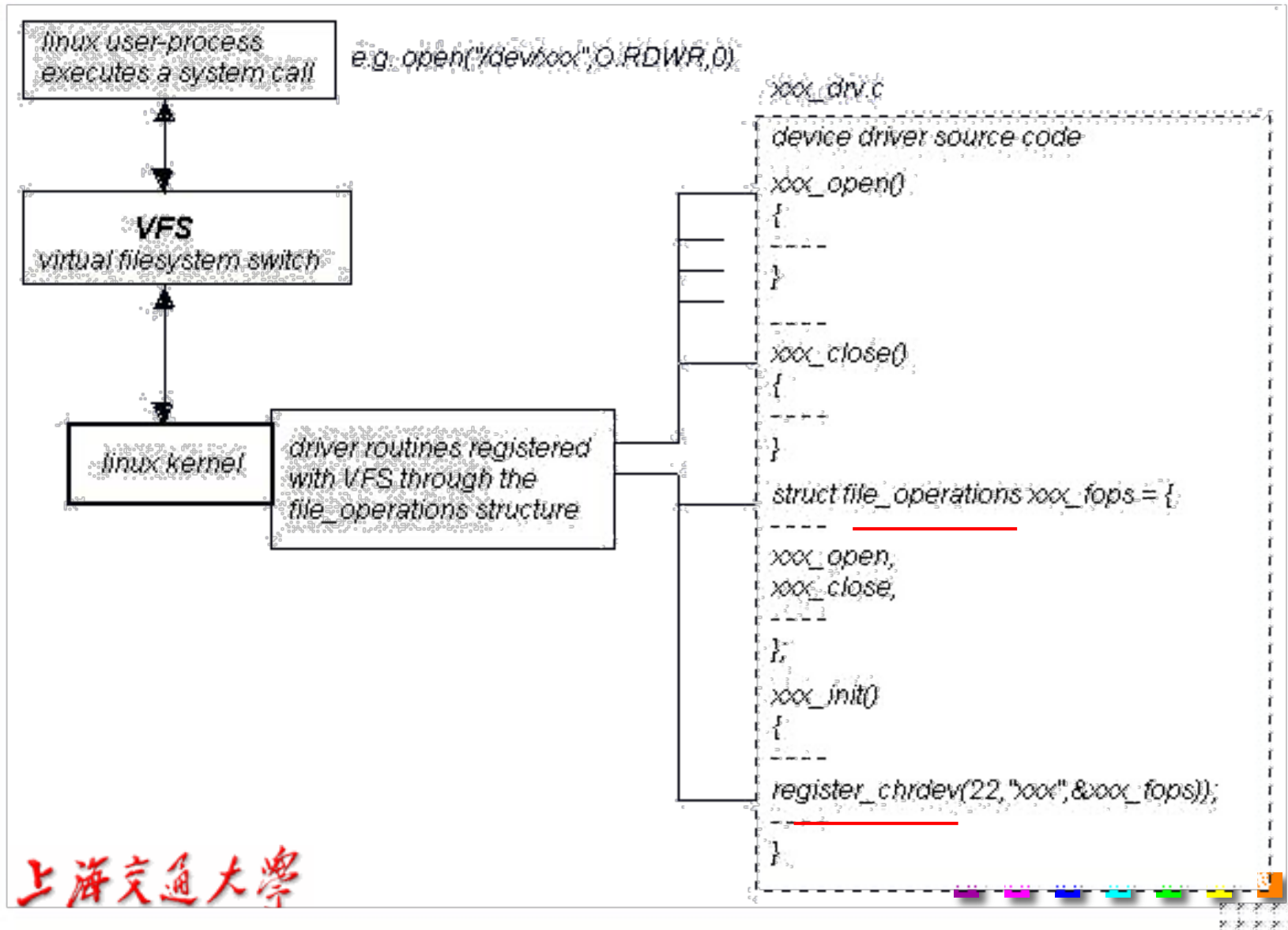


File operations...

lseek	Changes current r/w pos in a file, Returns the new position
read	Used to retrieve data from the device
write	Sends data to the device.
readdir	NULL for device, Used for Filesystems
poll	Inquire if a device is readable or writable or in some special state
ioctl	issue device-specific commands e.g. Format a floppy disk
mmap	request a mapping of device memory to a process's addr space
open	First operation, not needed for Device Drivers

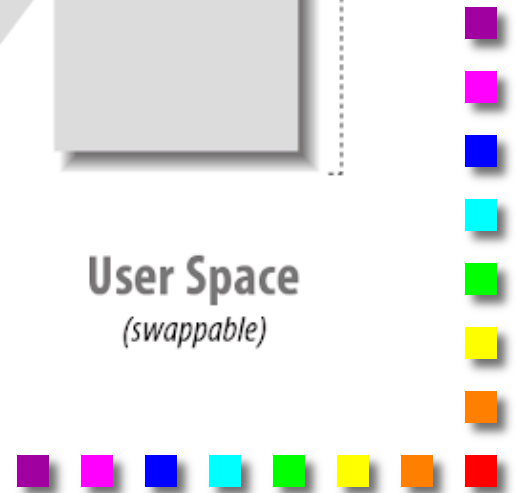
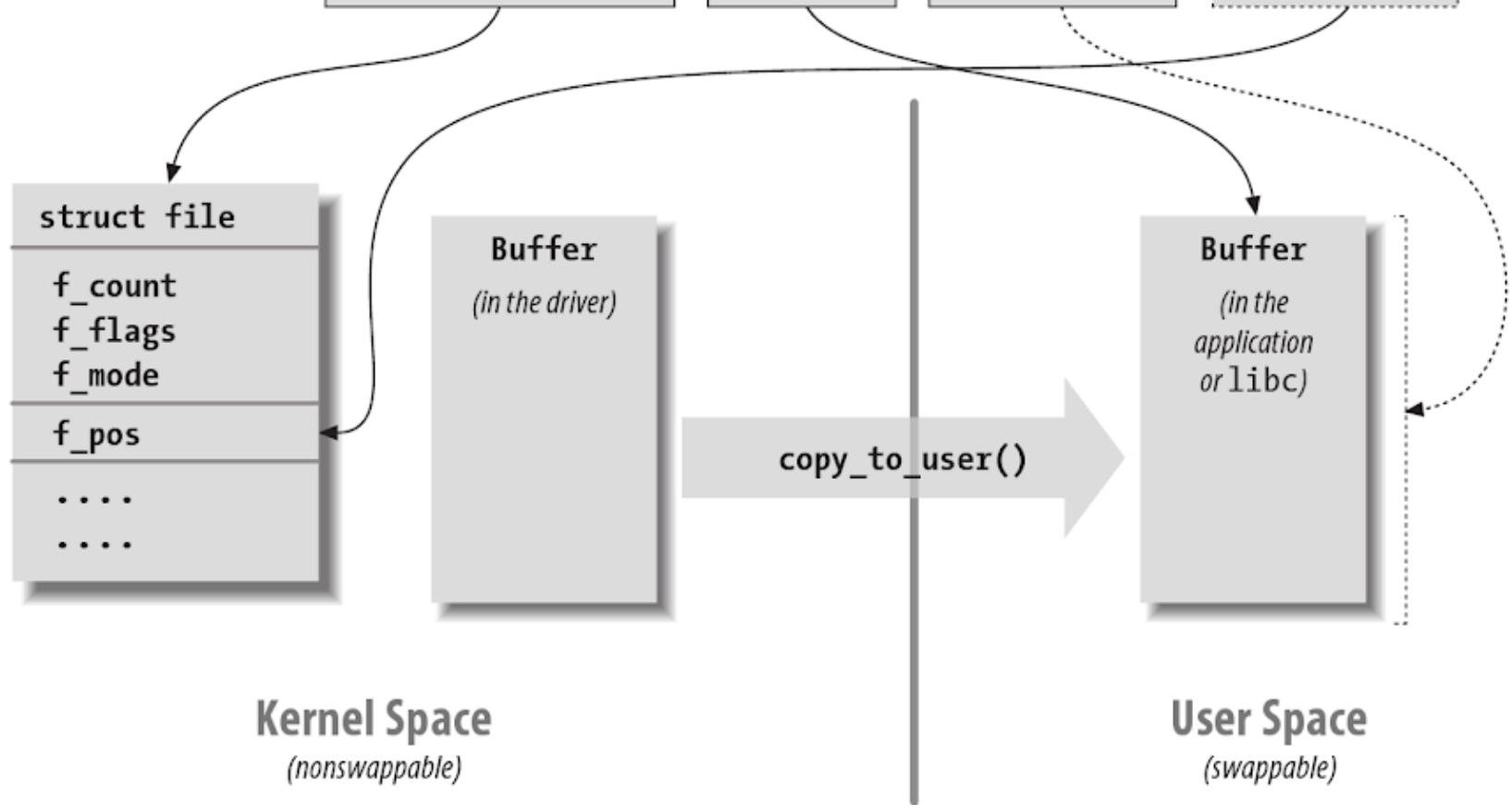


Mapping calls to dev functions



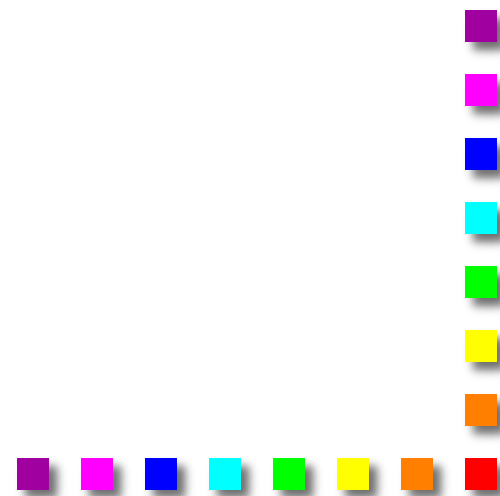
Read() and write()

```
ssize_t dev_read(struct file *file, char *buf, size_t count, loff_t *ppos);
```



Outline

- What are Device Drivers
- Character Device Drivers
- Block Device Drivers
- DMA
- Network Device Drivers
- Timer

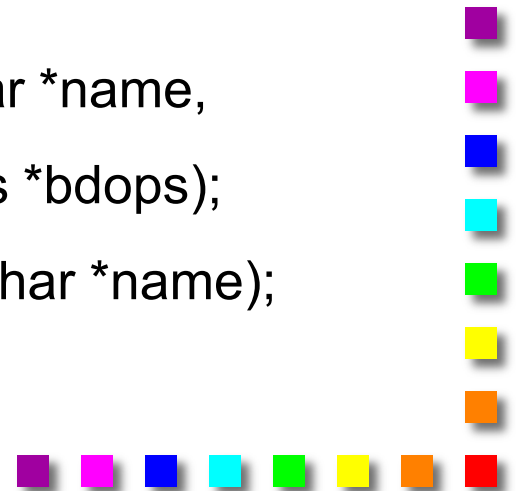


Registering a device

- Block drivers: identified by major numbers
 - Block major numbers are entirely distinct from char major numbers
 - A block device with a major number **can coexist with a char device using the same major number**
- Commands to register

```
int register_blkdev (unsigned int major, const char *name,  
                    struct block_device_operations *bdops);
```

```
int unregister_blkdev (unsigned int major, const char *name);
```

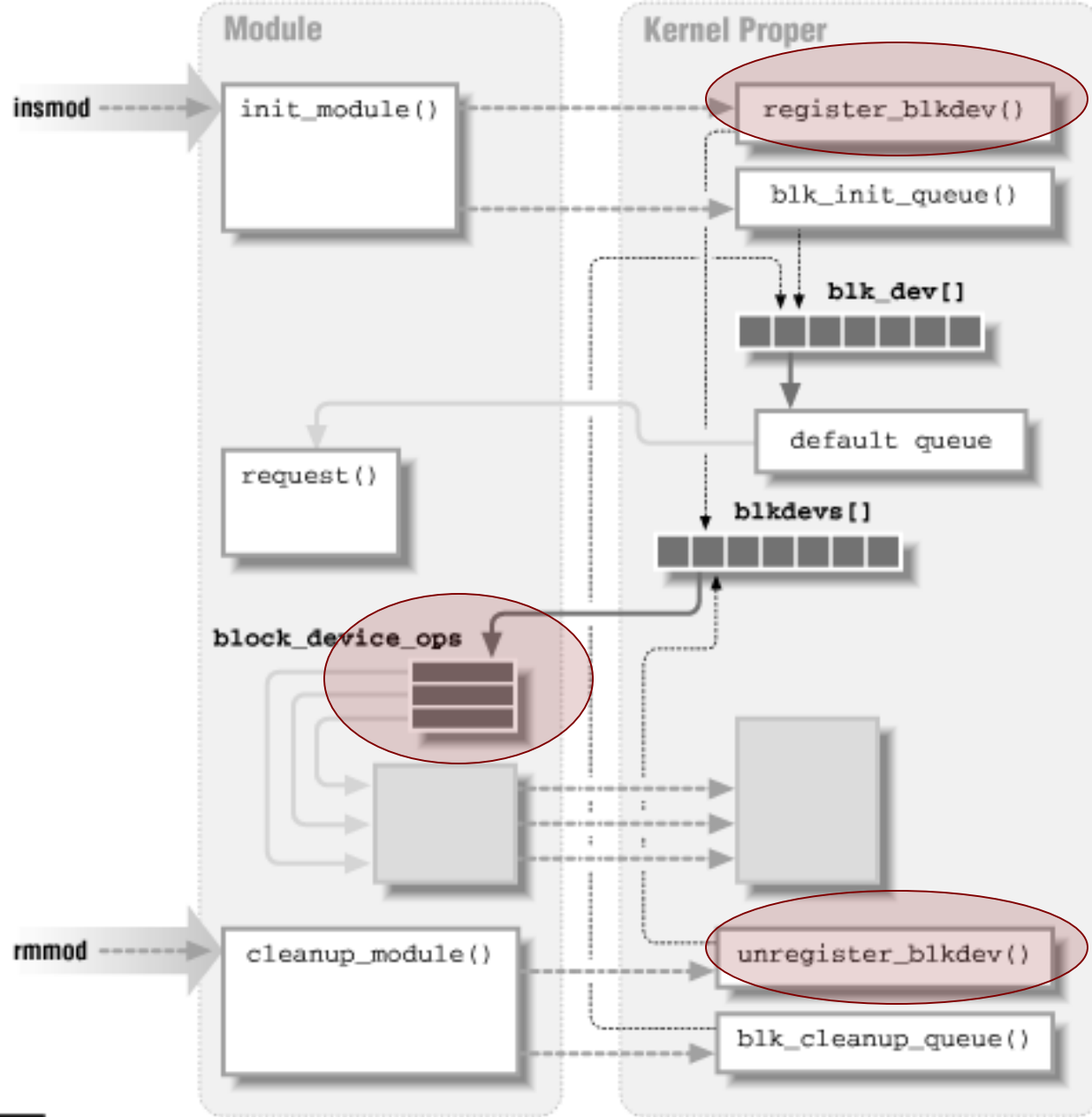


Block Device Operations

```
struct block_device_operations {  
    int (*open) (struct inode *inode, struct file *filp);  
    int (*release) (struct inode *inode, struct file *filp);  
    int (*ioctl) (struct inode *inode, struct file *filp, unsigned command,  
        unsigned long argument);  
    int (*check_media_change) (kdev_t dev);  
    int (*revalidate) (kdev_t dev); };
```

- There are **no** read or write operations provided in the `block_device_operations` structure.
- All I/O to block devices is normally **buffered** by the system





KEY



One Function

Multiple Functions



Data



Function call

Function pointer



Data pointer

Assignment to data



上海

Block Devices: How I/O is done

■ Define *request* function

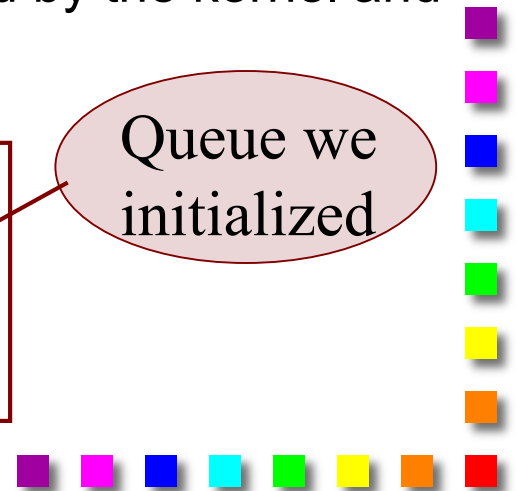
- Request function is with the **queue** of pending I/O operations for the device.
- There is one such queue for each major number by default.
- A block driver must initialize that queue with *blk_init_queue*.
- Queue accessed by major number: **BLK_DEFAULT_QUEUE**(major)
 - This macro looks into a global array of **blk_dev_struct** structures called **blk_dev**, which is maintained by the kernel and indexed by major number

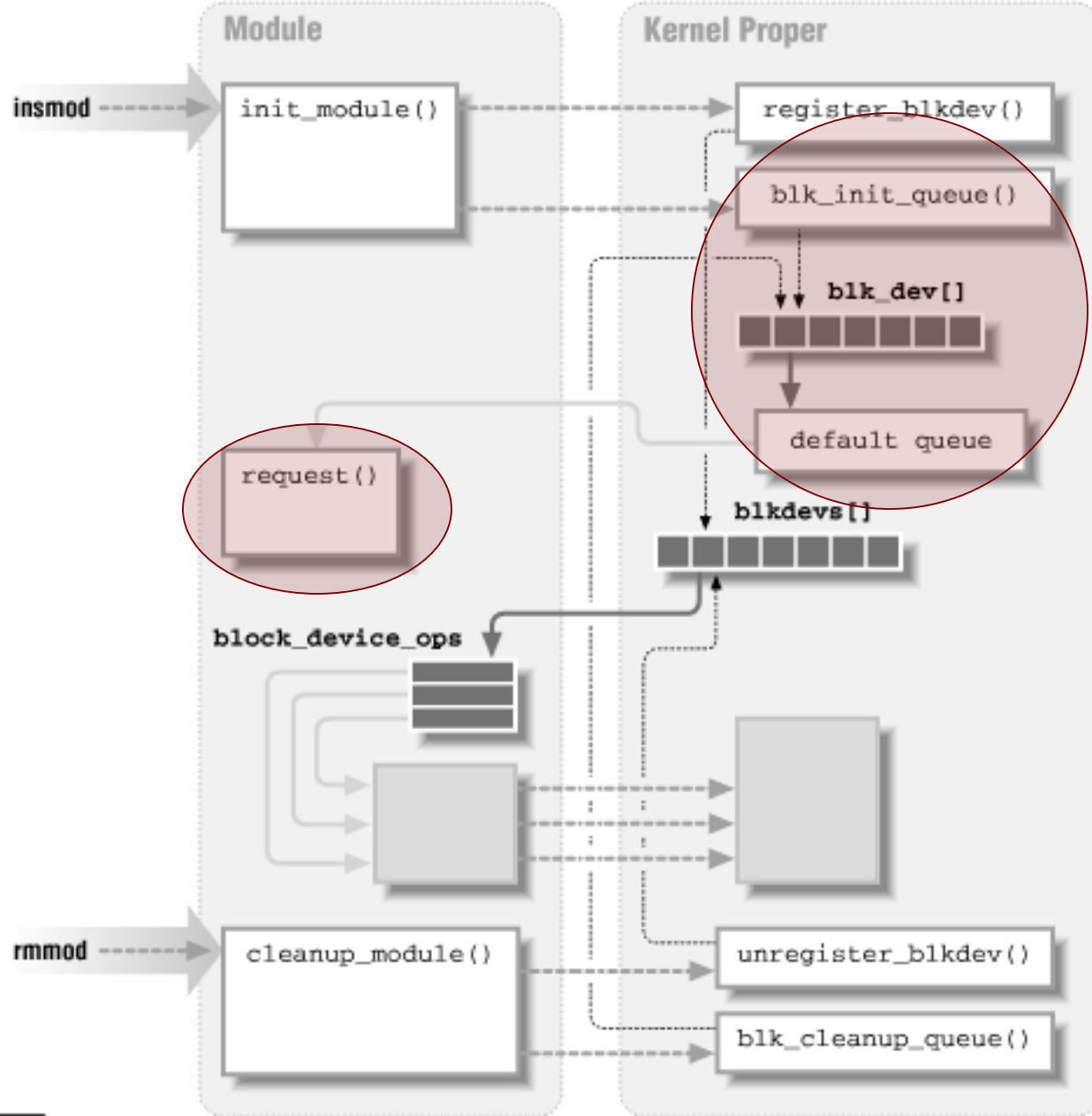
```
struct blk_dev_struct  
{ request_queue_t request_queue;  
  queue_proc *queue;  
  void *data; };
```

Queue we
initialized



上海交通大学





KEY



One Function

Multiple Functions



Data



Function call

Function pointer



Data pointer

Assignment to data

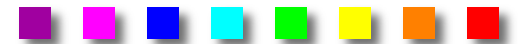


上海

Information from Kernel

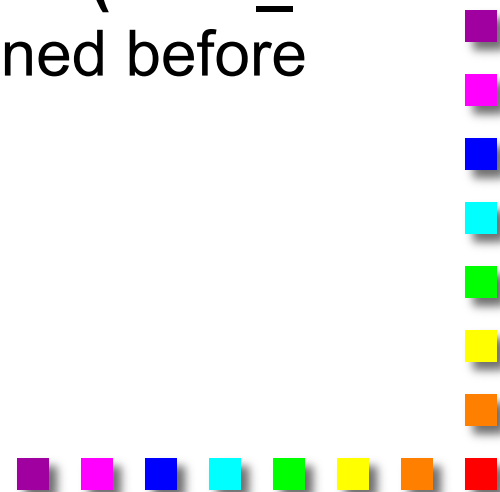
- In addition to blk_dev, several other global arrays hold information about block drivers (drivers/block/ll_rw_block.c).

int blk_size[][];	describes the size of each device
int blksize_size[][];	size of the block used by each device, in bytes
int read_ahead[];	number of sectors to be read in advance by the kernel
int max_sectors[][];	array limits the maximum size of a single request
int max_segments[];	number of individual segments that could appear in a clustered request



Header File: blk.h

- All block drivers must include the header file **<linux/blk.h>**
- This file defines much of the **common code** that is used in block drivers, and it provides functions for dealing with the **I/O request queue**
- **MAJOR_NR, DEVICE_NAME, DEVICE_NR (kdev_t device)** device specific fields must be defined before including



Request Function

■ The Request Queue

- When the kernel schedules a data transfer, it **queues the request** in a list, ordered in such a way that it maximizes system performance.
- The queue of requests is then passed to the driver's **request** function, which has the following prototype:
 - void **request_fn** (request_queue_t *queue);



What does *request* do?

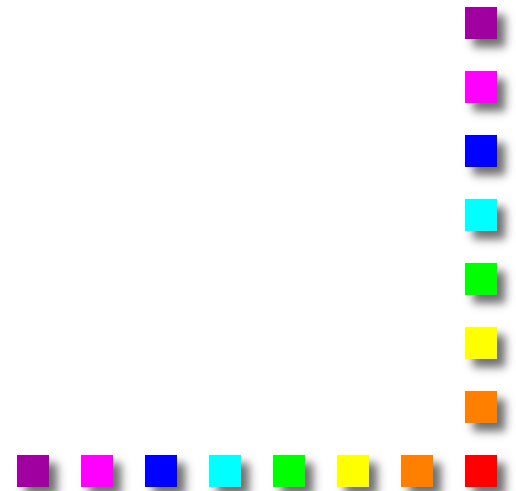
- 1) Checks **validity** of the request (INIT_REQUEST)
- 2) **Performs the actual data transfer** (The CURRENT variable (macro) can be used to retrieve the details of the current request)
- 3) **Cleans up the request** just processed. (*end_request*)
- 4) Loops back to the beginning, to **consume the next request**



Minimal request function

```
void sbull_request (request_queue_t *q)

{ while(1)
  { INIT_REQUEST;
    printk("<1>request %p: cmd %i sec %li (nr. %li)\n",
      CURRENT, CURRENT->cmd,
      CURRENT->sector,
      CURRENT->current_nr_sectors);
    end_request(1);
    /* success */ }
}
```



Data Transfer

- By accessing the fields in the request structure, usually by way of `CURRENT`, the driver can retrieve all the information needed to transfer data between the buffer cache and the physical block device
- `CURRENT` is just a pointer to **`blk_dev[MAJOR_NR].request_queue`**
- *Important Fields*
 - **`kdev_t rq_dev`** : The device accessed by the request
 - **`int cmd`** : Operation to be performed; Read or Write
 - **`unsigned long sector`**: The number of the first sector to be transferred in this request
 - **`char *buffer`**: The area in the buffer cache to which data should be written/ read



Making Accesses Faster

■ Clustering

- Clustering of requests to adjacent sectors on the disk.
- Modern filesystems will attempt to lay out files in consecutive sectors

=> requests to adjacent parts of the disk are common

■ “Elevator” algorithm

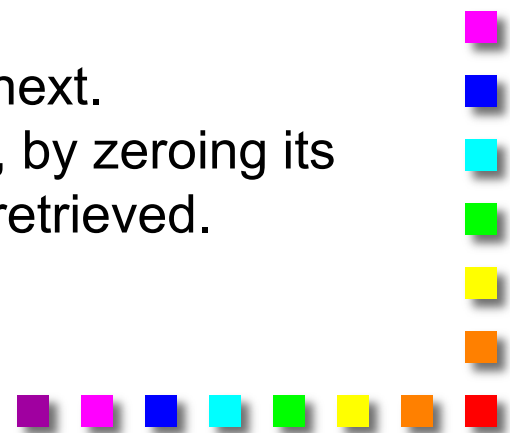
- The kernel tries to keep the disk head moving in the same direction for as long as possible

=> minimize seek times and increase throughput



How Clustering Works

- Block driver must **look directly at the list of buffer_head** structures attached to the request.
- This list is pointed to by `CURRENT->bh`; subsequent buffers can be found by following the `b_reqnext` pointers in each `buffer_head` structure.
- Algorithm
 - 1) Arrange to transfer the data block at address `bh->b_data`, of size `bh->b_size` bytes.
The direction of the data transfer is `CURRENT->cmd` (READ/ WRITE).
 - 2) Retrieve the next buffer head in the list: `bh->b_reqnext`.
Then detach the buffer just transferred from the list, by zeroing its `b_reqnext` -- the pointer to the new buffer you just retrieved.



How Clustering Works

- 3) Update the request structure to reflect the I/O done with the buffer that has just been removed.
Both `CURRENT->hard_nr_sectors` and `CURRENT->nr_sectors` should be decremented by the number of sectors (not blocks) transferred from the buffer.
- 4) The sector numbers `CURRENT->hard_sector` and `CURRENT->sector` should be incremented by the same amount.
- 5) Loop back to the beginning to transfer the next adjacent block.

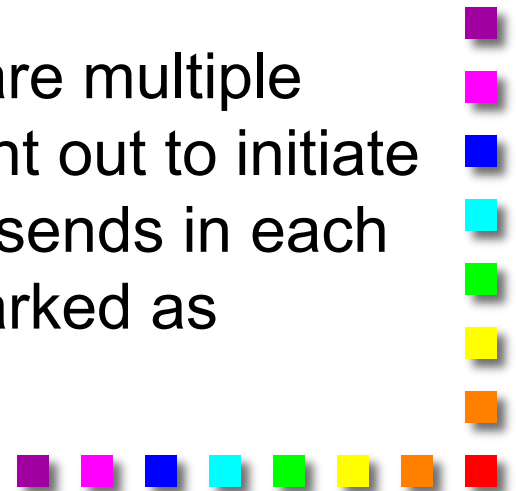
After I/O completes notify the kernel by calling the buffer's I/O completion routine: `bh->b_end_io(bh, status);`



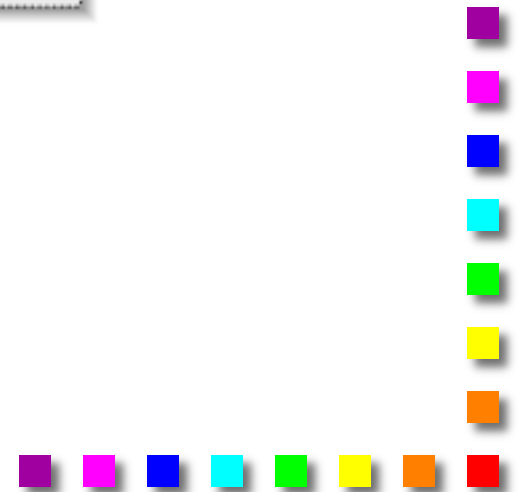
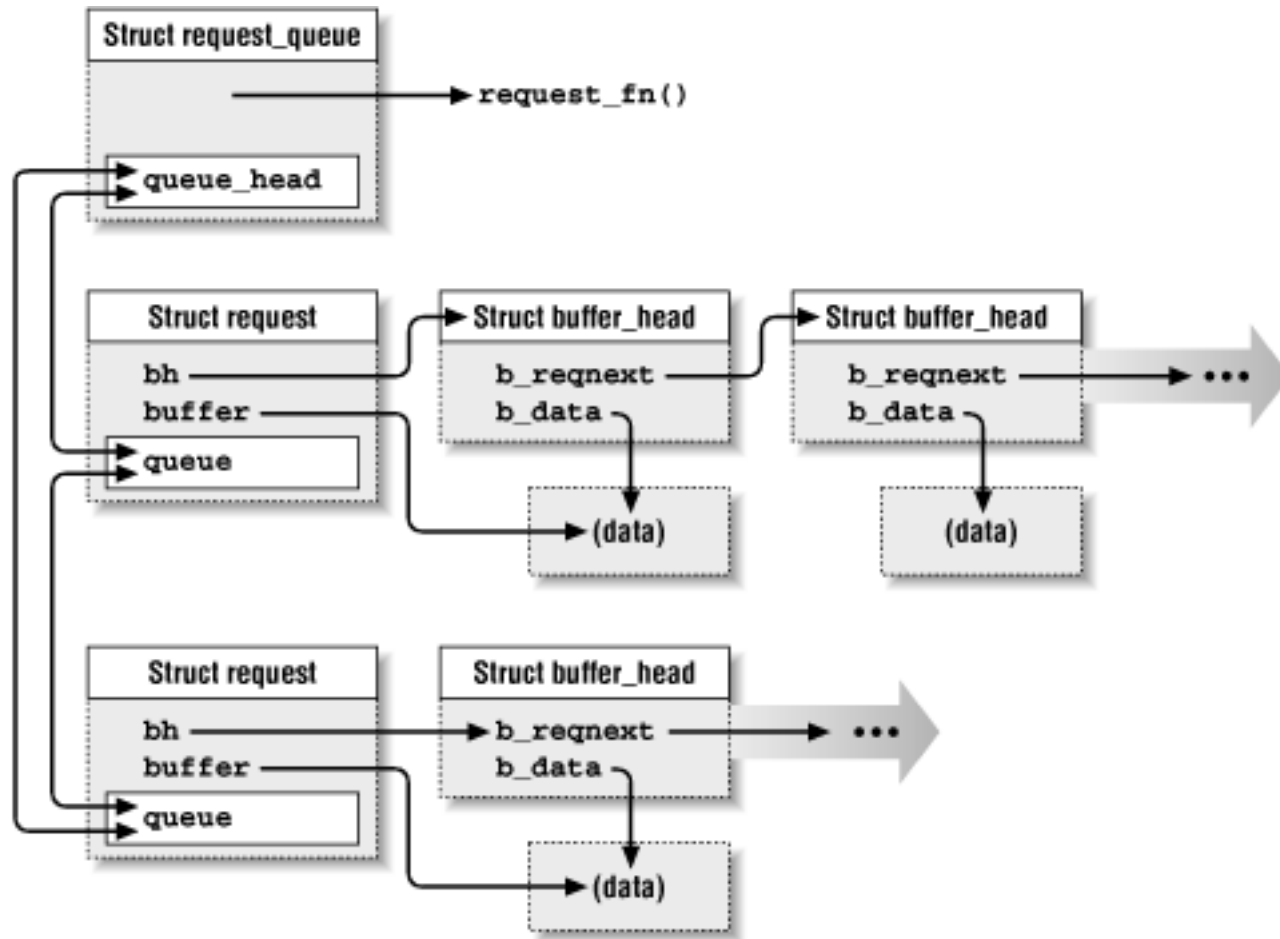
Making Accesses Faster

■ Scatter Gather

- The "**scatter**" part means that when there are multiple blocks to be written all over a disk
 - Example: one command is sent out to initiate writing to different sectors, reducing the **overhead involved in negotiation from $O(n)$ to $O(1)$** , where n is the number of blocks or sectors to write.
- "**Gather**" part means that when there are multiple blocks to be read, one command is sent out to initiate reading all the blocks, and as the disk sends in each block, the corresponding request is marked as satisfied with **end_request(1)**.

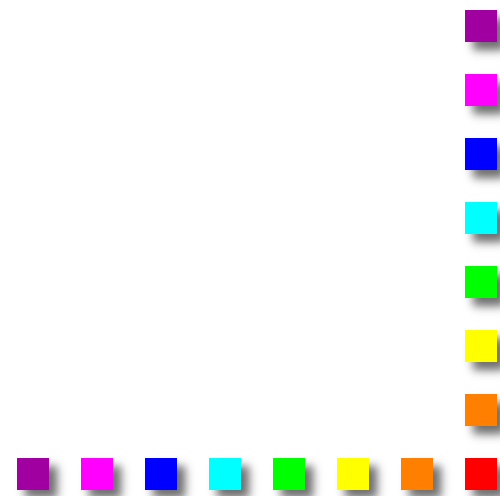


Buffers in the I/O Request Queue



Outline

- What are Device Drivers
- Character Device Drivers
- Block Device Drivers
- DMA
- Network Device Drivers
- Timer



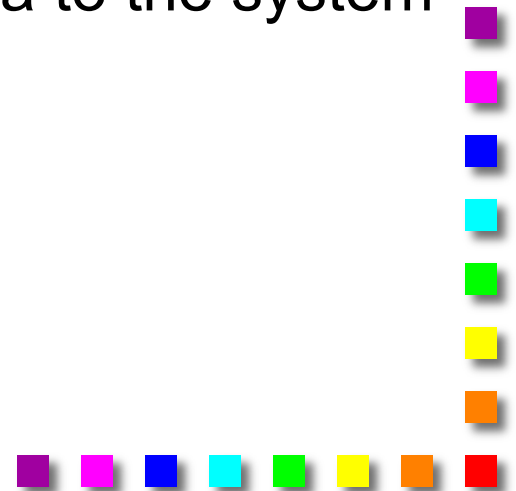
What is DMA

- DMA is the hardware mechanism that allows peripheral components to transfer their I/O data directly to and from main memory **without** the need for the system processor to be involved in the transfer.
- Greatly increase **throughput** to and from a device
- Device driver needs to be able to correctly set up the **DMA transfer** and **synchronize** with the hardware
- DMA is very **system dependent**



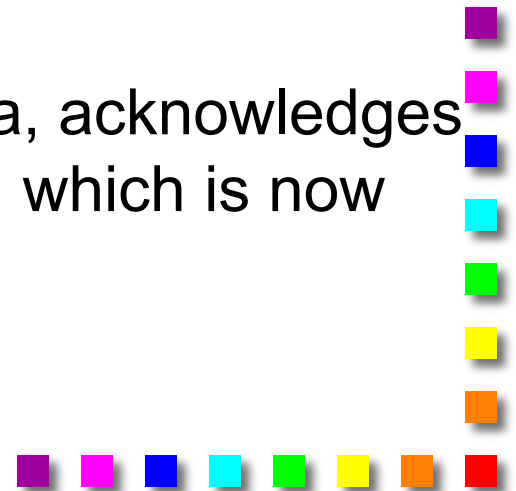
When is DMA needed

- Data transfer can be triggered in two ways:
 - Software asks for data (via a function such as *read*)
 - Hardware asynchronously pushes data to the system



Case I: Software asks for data

- When a process calls *read*, the driver method allocates a DMA buffer and instructs the hardware to transfer its data. The process is put to sleep.
- The hardware writes data to the DMA buffer and raises an interrupt when it is done.
- The interrupt handler gets the input data, acknowledges the interrupt, and awakens the process, which is now able to read data.



Case II: Asynchronous DMA

- The hardware raises an interrupt to announce that new data has arrived.
- The interrupt handler allocates a buffer and tells the hardware where to transfer its data.
- The peripheral device writes the data to the buffer and raises another interrupt when it is done.
- The handler dispatches the new data, wakes any relevant process, and takes care of housekeeping.



Case III: Network Cards

- These cards often expect to see a circular buffer (often called a *DMA ring buffer*) established in memory shared with the processor
- Each incoming packet is placed in the next available buffer in the ring, and an interrupt is signaled.
- The driver then passes the network packets to the rest of the kernel, and places a new DMA buffer in the ring.



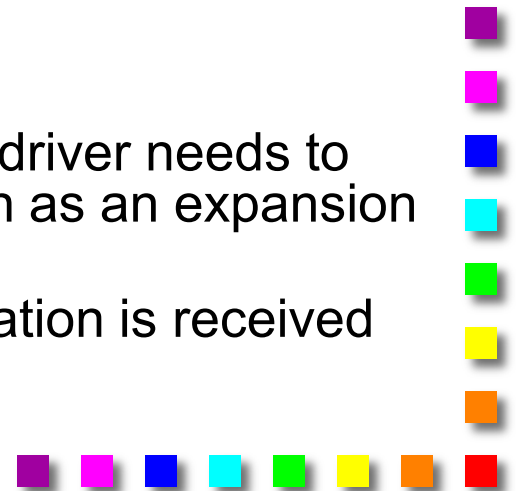
Allocating DMA Buffers

- The main problem with the DMA buffer is that when it is **bigger than one page**
- It must occupy **contiguous pages** in physical memory because the device transfers data using the ISA or PCI system bus, both of which carry physical addresses.



Bus Addresses

- A device driver using DMA has to talk to hardware connected to the interface bus, which uses **physical addresses**, whereas program code uses virtual addresses.
- *Solution*
*unsigned long virt_to_bus(volatile void * address);*
*void * bus_to_virt(unsigned long address);*
- *virt_to_bus* conversion must be used when the driver needs to send address information to an I/O device (such as an expansion board or the DMA controller)
- *bus_to_virt* must be used when address information is received from hardware connected to the bus.



DMA Mappings

- A *DMA mapping* is a combination of
 - Allocating a DMA buffer
 - Generating an address for that buffer that is accessible by the device
- *Mapping Registers* (virtual memory for peripherals)
 - 1) Peripherals have a relatively **small**, dedicated range of addresses to which they may perform DMA
 - 2) Those addresses are **remapped**, via the mapping registers, into system RAM.
 - 3) Have ability to make several distributed pages appear **contiguous** in the device's address space.



DMA Mappings

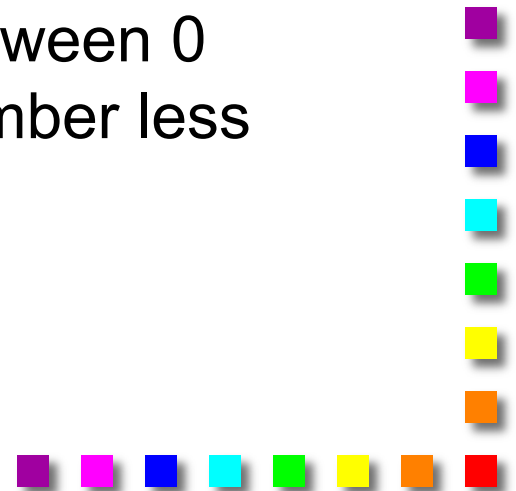
■ *Bounce Buffer*

- Bounce buffers are created when a driver attempts to perform DMA on an address that is **not reachable** by the peripheral device
 - eg., a high-memory address
- Data is then **copied to and from** the bounce buffer as needed.



Registering DMA Usage

- `int request_dma(unsigned int channel, const char *name);`
- `void free_dma(unsigned int channel);`
- The channel argument is a number between 0 and 7 or, more precisely, a positive number less than MAX_DMA_CHANNELS.



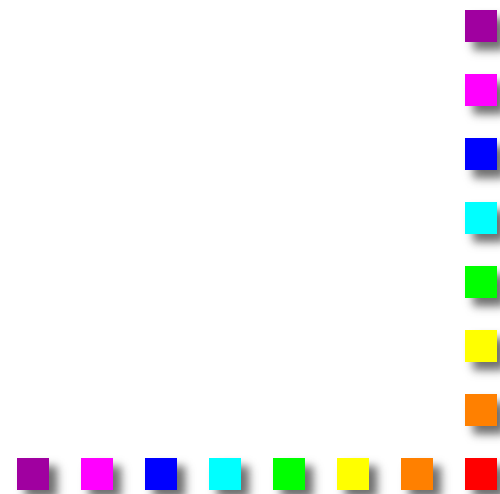
DMA: a shared Resource

- unsigned long ***claim_dma_lock()***
 - Acquires the DMA ***spinlock***.
 - This function also blocks interrupts on the local processor thus the return value is the usual "flags" value, which must be used when reenabling interrupts.
- void ***release_dma_lock***(unsigned long flags)



Outline

- What are Device Drivers
- Character Device Drivers
- Block Device Drivers
- DMA
- Network Device Drivers
- Timer



Network Device Drivers

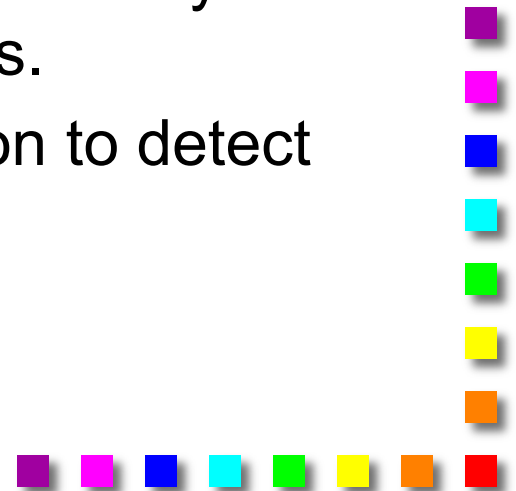
- Attaches a network subsystem to a network interface
- Difference from block devices – **Interacts with the outside world**
- Prepares the network interface for operation, transmission and reception of network frames
- Sets addresses, modifies transmission parameters and maintaining traffic statistics



Network Device Drivers

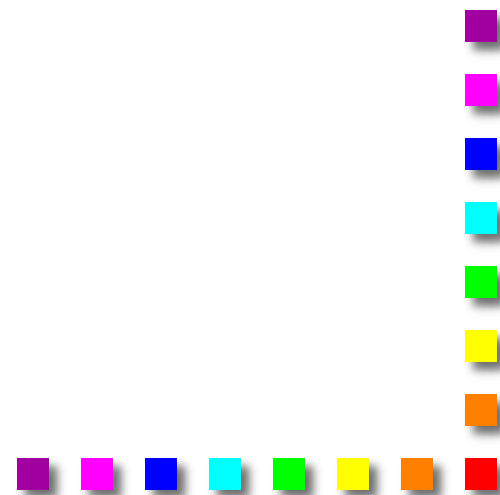
Transmission Timeouts for Network Devices

- Hardware may fail – drivers must be prepared.
- Problem of missing Interrupts - solved by using a mass of timers.
- Any network system is a complicated assembly of state machines controlled by a mass of timers.
- Networking code level – the best position to detect transmission timeouts.



Outline

- What are Device Drivers
- Character Device Drivers
- Block Device Drivers
- DMA
- Network Device Drivers
- Timers



Timer Interrupt

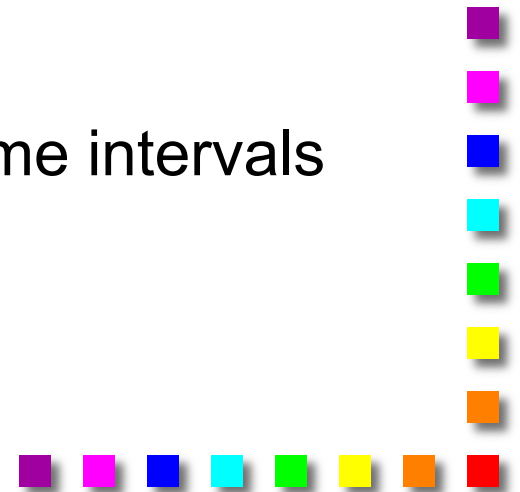
- The mechanism used by the kernel to keep track of time intervals
- Generated by the system's timing hardware at regular intervals
 - Interval is set by the kernel according to the value of HZ, which is an architecture-dependent value defined in `<linux/param.h>`
 - Current Linux versions define HZ to be 100 for most platforms.



Mechanism

■ Jiffies

- The number of clock ticks since the computer was turned on
- Declared in `<linux/sched.h>` as unsigned long volatile
- Generally sufficient for measuring time intervals (according to the least count)



Counter Register

- Counter register is steadily incremented once at each clock cycle.
- Platform dependent
 - may or may not be writable
 - may or may not be readable from user space
 - 64 or 32 bits wide
 - Used for measuring very short time lapses with precision



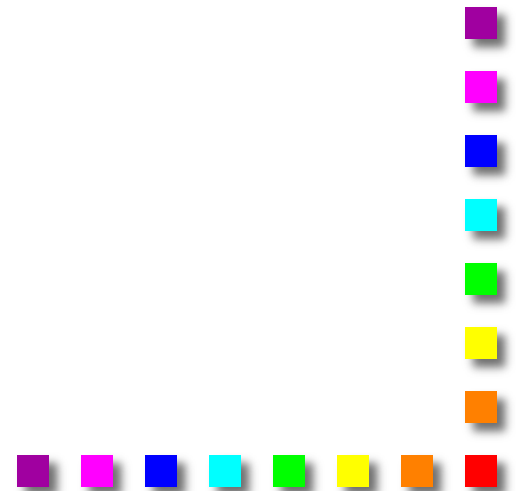
TSC (timestamp counter)

- Introduced in x86 processors with the Pentium and present in all CPU designs ever since
- 64-bit register that counts CPU clock cycles
- Can be read from both kernel space and user space



Scheduling tasks at a later time without using interrupts

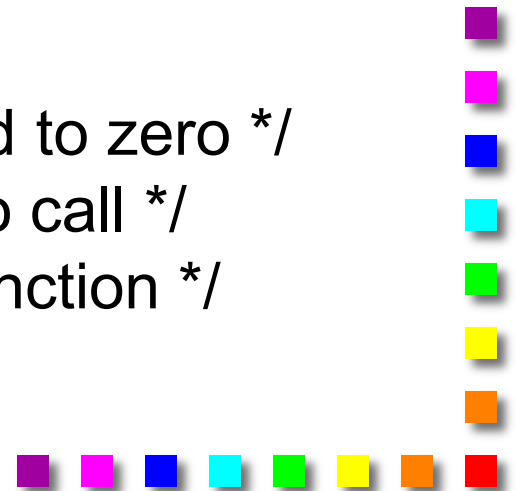
- Three interfaces are available
 - Task queues
 - Tasklets
 - Kernel timers



Task queues

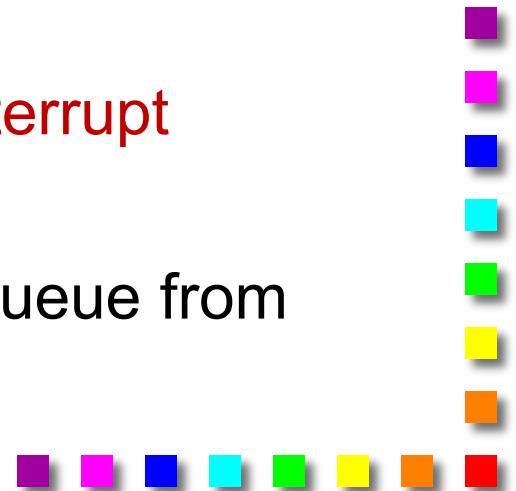
- It is a **list of tasks**, each task being represented by a function pointer and an argument
- A queue element is described by the following structure, copied directly from **<linux/tqueue.h>**:

```
struct tq_struct {  
    struct tq_struct *next;  
    int sync;           /* must be initialized to zero */  
    void (*routine)(void *); /* function to call */  
    void *data;         /* argument to function */  
};
```



Task queues

- Different queues are run at different times, but they are always run when the kernel has no other pressing work to do
- Almost never run when the process that queued the task is executing
- Often run as the result of a software interrupt
- A task can requeue itself in the same queue from which it was run



Predefined task queues

Driver can use only three :

■ The scheduler queue

- unique among the predefined task queues in that it runs in process context, implying that the tasks it runs have a bit more freedom in what they can do

■ tq_timer

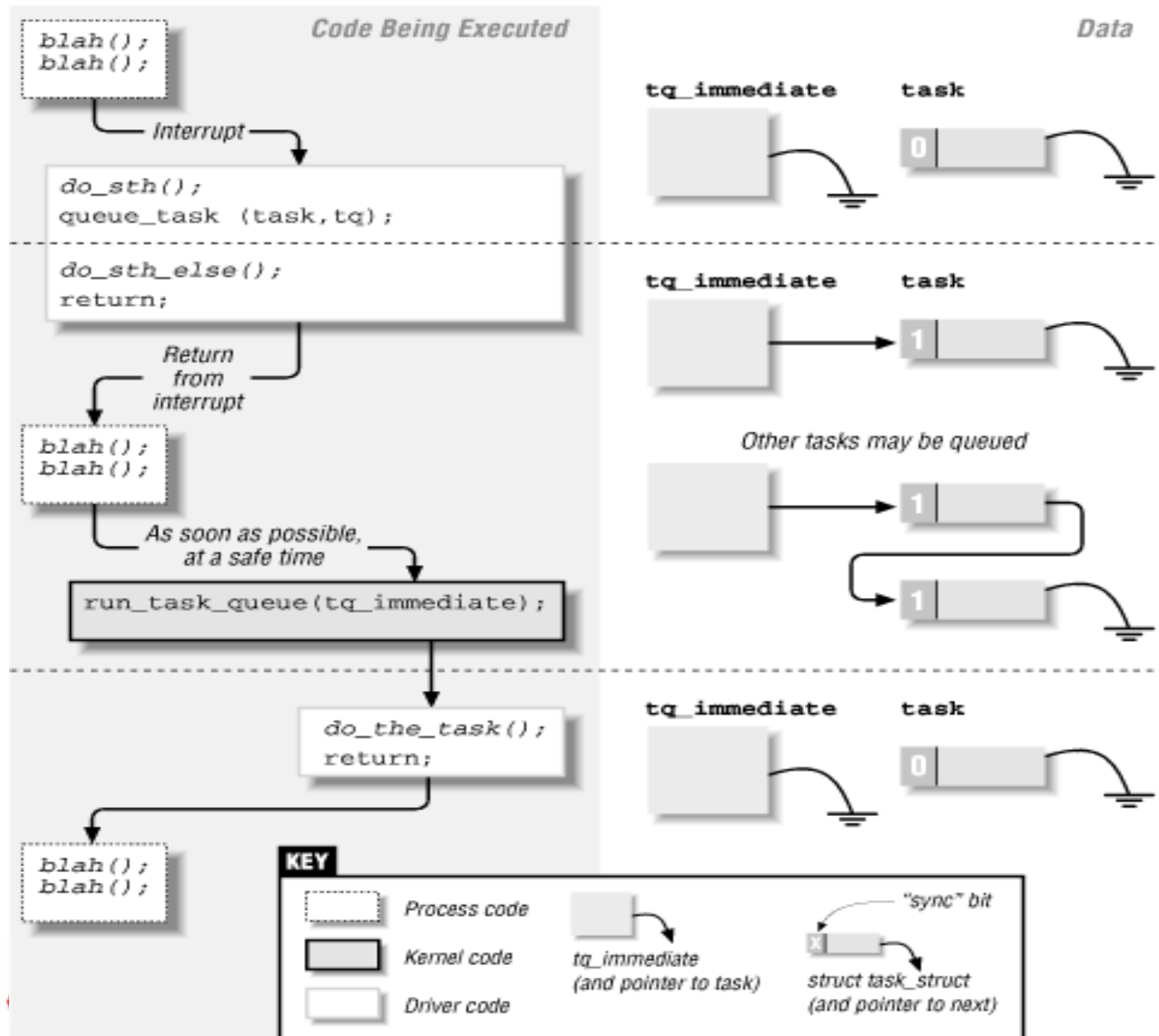
- run by the timer tick. Because the tick (the function *do_timer*) runs at interrupt time, any task within this queue runs at interrupt time as well.

■ tq_immediate

- The immediate queue is run as soon as possible, either on return from a system call or when the scheduler is run, whichever comes first. The queue is consumed at interrupt time.



Task queues



Tasklets

- Way of deferring a task until a safe time, and they are always run in interrupt time
- Tasklets will be run only *once*, even if scheduled multiple times
- May be run in *parallel* with other tasklets on SMP systems
- Each tasklet has associated with it a function that is called when the tasklet is to be executed



Tasklets

- **DECLARE_TASKLET** (name, function, data);
 - Declares a tasklet with the given name; when the tasklet is to be executed, the given function is called with the (unsigned long) data value
- **DECLARE_TASKLET_DISABLED** (name, function, data);
 - Declares a tasklet as before, but its initial state is "disabled," meaning that it can be scheduled but will not be executed until enabled at some future time.



Kernel Timers

- Timers are used to **schedule execution of a function** (a timer handler) at a particular time in the future
- We can specify exactly when in the future the function will be called
- You register your function **once**, and the kernel calls it when the timer expires
- Function registered in a kernel timer is **executed only once**



Kernel Timers

- Once a **timer_list structure** is initialized, *add_timer* inserts it into a sorted list, which is then polled more or less 100 times per second
- **Race conditions**
 - the timer expires at just the right time, even if the processor is executing in a system call
 - Any data structures accessed by the timer function should be protected from **concurrent access**
 - To avoid race conditions while deleting the timers, one must use *del_timer_sync* instead of *del_timer*.



End.

All you guys have a good score. 😊

